

Performance Basics

Making a webpage load faster and feel smoother

Introduction

Every webpage takes some time to reach a visitor's screen and become usable. Performance is about how quickly that happens and how smooth the experience feels while it's happening. Two things affect this the most: how the images on a page are handled, and the order in which the browser is told to load everything. Small changes can make a page feel noticeably faster, even without changing what the page actually contains.

1. Image Optimization

Images are usually the heaviest files on a webpage. A single photo can be larger in size than the rest of the page put together. Image optimization means preparing and loading images in a way that keeps the page as light and fast as possible.

1.1 width and height attributes

The width and height attributes tell the browser exactly how much space an image will occupy on the page, even before the image itself has finished loading. Without this information, the browser doesn't know how tall or wide the image will be, so it leaves no space for it at first. The moment the image finally loads, everything below it suddenly gets pushed down. This sudden shifting of content is uncomfortable to read and is one of the most common annoyances on slow-loading pages.

```

```

In short: Setting width and height lets the browser reserve the correct space in advance, so the page doesn't jump around while it loads.

1.2 The alt attribute

The alt attribute is a short text description of what an image shows. If the image cannot load for any reason, this text appears in its place. It is also read aloud by screen readers, which are tools used by people who are blind or have low vision to browse the web. Search engines also read this text to understand what an image contains, since they cannot see the image itself.

```

```

In short: A clear alt description makes an image useful even when it can't be seen — by a person, a screen reader, or a search engine.

1.3 The loading attribute (lazy loading)

By default, a browser tries to load every image on a page the moment the page opens, even images far down at the bottom that a visitor might never scroll down to see. The loading attribute, when set to "lazy", tells the browser to hold off on loading a particular image until

the visitor is about to scroll near it. This means the page has far less to load right at the start, so the visible part of the page appears sooner.

```

```

In short: Use `loading="lazy"` on images further down a page. The very first images a visitor sees should load normally, since they need to appear right away.

1.4 srcset and sizes attributes

Not every visitor is viewing a page on the same size of screen. A large, detailed image looks great on a desktop monitor but is unnecessarily heavy for a small phone screen. The `srcset` attribute gives the browser several versions of the same image, each at a different size, and the browser automatically picks whichever one best fits the visitor's screen. The `sizes` attribute gives the browser extra detail about how large the image will actually be displayed, helping it choose more accurately.

```

```

In short: `srcset` lets the browser choose the right-sized image for each visitor, instead of sending everyone the same large file.

1.5 The picture element

Sometimes the goal isn't just a smaller version of the same image, but a completely different image depending on the screen size — for example, a wide banner for a desktop screen and a cropped, square version for a phone. The `picture` element allows several source options to be listed, each with its own condition, and the browser picks whichever one matches. If none of the conditions match, it simply falls back to the regular `img` tag placed inside it.

```
<picture>
  <source media="(max-width: 600px)" srcset="banner-mobile.jpg">
  <source media="(min-width: 601px)" srcset="banner-desktop.jpg">
  
</picture>
```

In short: `picture` is used when the image itself should change for different screens, not just its size.

1.6 Image formats

Different image formats are suited to different kinds of images:

- **JPEG** — best for photographs and images with lots of colour and detail. It compresses well, with only a very slight, usually unnoticeable, drop in quality.

- **PNG** — best for images that need a transparent background, or images with sharp edges and text, such as logos.
- **SVG** — used for icons, logos, and simple graphics made of shapes and lines. Unlike other formats, an SVG image can be resized to any size without ever looking blurry, because it is drawn using instructions rather than stored as a fixed grid of coloured dots.
- **WebP** — a newer format that produces a smaller file size than JPEG or PNG while keeping very similar quality. It is supported by all modern browsers.

1.7 Compression

Compression means reducing the file size of an image so that it takes up less space and loads faster.

- **Lossy compression** removes small bits of image data that are unlikely to be noticed, resulting in a much smaller file with a very slight drop in quality. JPEG typically works this way.
- **Lossless compression** makes the file smaller without removing any data at all, so the quality stays exactly the same as the original. PNG typically works this way.

Free tools such as TinyPNG or Squoosh can compress an image before it is uploaded to a website, without needing to change anything in the HTML code itself.

2. Loading Strategies

A loading strategy is the order and manner in which a browser loads the different pieces of a webpage — the HTML text, the images, and any scripts. Controlling this order can make a page feel much faster, even when the total amount of data being loaded stays exactly the same.

2.1 How a browser loads a page

When a browser opens a webpage, it reads through the HTML file from top to bottom, one piece at a time. Whenever it comes across a tag such as script, it usually pauses everything else until that particular file has finished downloading and running, before it continues reading the rest of the page. This means the position of a tag inside the HTML file can directly affect how quickly the rest of the page appears on screen.

2.2 Where a script tag is placed

If a script tag is placed inside the head section, the browser reaches it before it has read any of the visible content inside the body. It pauses everything else to download and run that script first, which can delay how soon the visitor sees anything on the screen at all.

```
<head>
  <script src="big-script.js"></script>
</head>
<body>
  <h1>Content the visitor is waiting to see</h1>
</body>
```

If the same script tag is instead placed near the bottom of the page, just before the closing body tag, the browser has already displayed all the visible content by the time it reaches the script. The visitor sees the page much sooner.

```
<body>
  <h1>Content the visitor is waiting to see</h1>
  <script src="big-script.js"></script>
</body>
```

In short: Placing scripts near the bottom of the page usually lets visible content appear sooner.

2.3 The async attribute

The async attribute can be added to a script tag. It tells the browser: keep reading and displaying the rest of the page while this script downloads quietly in the background. The moment the download finishes, pause briefly to run it, and then carry on. This is useful for a script that doesn't depend on anything else on the page and doesn't need to run in any particular order, such as a script that tracks how visitors use a page.

```
<script src="analytics.js" async></script>
```

2.4 The defer attribute

The defer attribute also downloads a script in the background without pausing the rest of the page. The difference is that a deferred script waits until the entire page has been fully read and displayed before it actually runs. If a page has several deferred scripts, they always run in the exact order they appear in the HTML. This is useful when a script needs the whole page to already exist before it runs, or needs to run in a specific order relative to other scripts.

```
<script src="app.js" defer></script>
```

2.5 async versus defer

Both attributes let a script download in the background instead of pausing the page, but they differ in exactly when the script is allowed to run:

- **A normal script tag** pauses the page completely — nothing else happens until it has finished downloading and running.
- **async** runs the script the instant it finishes downloading, even if that happens in the middle of the page still being displayed.
- **defer** waits until the whole page has finished being displayed, and then runs the scripts in the exact order they were written.

***In short:** Use `async` for independent scripts where timing doesn't matter. Use `defer` when a script needs the full page ready first, or needs to run in a set order.*

2.6 preload and preconnect

These are hints placed in the head section of a page that tell the browser about something it is going to need soon, before it has actually reached that part of the page.

- **preload** tells the browser that a specific file, such as an image or a font, will be needed soon, so it should start downloading it immediately instead of waiting until it is reached in the page.
- **preconnect** tells the browser that it will soon need to communicate with another website or server, such as one hosting fonts, so it should set up that connection in advance. Setting up a connection ahead of time saves a small amount of delay later.

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preload" href="hero-image.jpg" as="image">
```

2.7 Caching

Caching means storing a copy of a file so that it doesn't need to be downloaded again the next time it is needed. When a browser loads a webpage, it keeps copies of files such as images on the visitor's own device. The next time the same visitor opens the same page, the browser can use these saved copies straight away instead of downloading everything again from the internet. This is why returning to a website a second time usually feels much faster than the first visit.

3. Quick Summary

Concept	What it does
width / height	Reserves space for an image before it loads, so the page doesn't jump around.
alt	Describes an image in text form for accessibility and search engines.
loading="lazy"	Delays loading an image until the visitor scrolls near it.
srcset / sizes	Lets the browser pick the right-sized image for each screen.
picture	Lets the browser pick a completely different image for different screens.
Image formats	JPEG for photos, PNG for transparency, SVG for icons, WebP for smaller files.

Compression	Shrinks an image's file size, either with (lossy) or without (lossless) any quality loss.
Script placement	Scripts near the bottom of the page let visible content appear sooner.
async	Downloads a script in the background and runs it as soon as it's ready.
defer	Downloads a script in the background and runs it after the page is fully ready, in order.
preload	Tells the browser to start downloading a needed file early.
preconnect	Tells the browser to set up a connection to another server early.
Caching	Saves a copy of a file so it doesn't need to be downloaded again next time.

4. Practice

Exercise 1: Lazy loading images

1. Create a webpage with several images placed one below another.
2. Add width and height attributes to every image.
3. Add loading="lazy" to every image except the very first one.
4. Open the page in a browser, open developer tools, and switch to the Network tab.
5. Reload the page and notice that only some images are requested immediately.
6. Scroll down slowly and watch new image requests appear as the page gets closer to them.